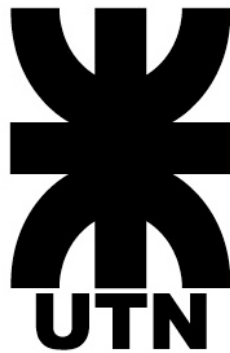


Trabajo Práctico Integrador

Programación I

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

ALUMNO: Gerardo Ocampo



Universidad Tecnológica Nacional (UTN)
Tecnatura Universitaria en Programación a Distancia
2026

Fecha de entrega: 15/06/26

ÍNDICE

1. MARCO TEÓRICO	3
2. DECISIONES TÉCNICAS Y ARQUITECTURAS	4
3. DIFICULTADES ENCONTRADAS Y RESOLUCIONES.....	5
4. CONCLUSIONES	6
5. BIBLIOGRAFÍA.....	6

1. MARCO TEÓRICO

El desarrollo de este sistema se basa en la integración de estructuras de datos y principios de persistencia en Python.

Estructuras de Datos:

Se utilizó una lista de diccionarios como estructura central. Las listas permiten almacenar una colección ordenada de elementos mutables, mientras que los diccionarios facilitan la representación de entidades (países) mediante pares clave-valor, mejorando la legibilidad y el acceso a los datos por nombre de campo en lugar de índices numéricos.

Modularización:

El código se organiza en funciones, donde cada una cumple una tarea específica y única, siguiendo el principio de una función = una responsabilidad. Esto facilita el mantenimiento y la escalabilidad del software.

Persistencia de Datos:

Se implementó mediante el manejo de archivos CSV utilizando la biblioteca nativa csv.

El sistema diferencia claramente entre la **Memoria RAM** (volátil y de alta velocidad) y el **Almacenamiento Permanente** (Disco Duro o SSD).

- **Ciclo de Sincronización:** Para ganar velocidad de procesamiento, los datos se cargan del disco a la RAM al iniciar la sesión (`cargar_datos`) y se vuelcan nuevamente al disco al finalizar (`guardar_datos`), garantizando que la información sobreviva al cierre del programa.
- **Formato CSV:** Se seleccionó el formato **Valores Separados por Comas (CSV)** por su ligereza y su alta **interoperabilidad**, permitiendo que los datos generados sean legibles por aplicaciones externas como Excel o bases de datos relacionales. El uso de las clases descritas (`DictReader` y `DictWriter`) permite un mapeo automático entre las filas del archivo y los diccionarios de Python, protegiendo la integridad de las columnas. Estos diccionarios se encuentran en la memoria RAM, asegurando que los cambios realizados durante la ejecución se conserven de forma permanente en el disco duro.

Manejo de Errores (Robustez):

Un pilar fundamental del proyecto fue la robustez, definida como la capacidad del sistema para anticipar y gestionar situaciones problemáticas sin detener su ejecución

- **Bloques Try-Except:** Se implementan para capturar excepciones de tiempo de ejecución, como ingresos de tipo incorrecto (`ValueError`) o fallos en el acceso a recursos físicos (`FileNotFoundError`).
- **Sanitización de Entradas:** El sistema aplica técnicas de limpieza de datos mediante los métodos `.strip()` para eliminar caracteres invisibles (espacios o saltos de línea) y `.lower()` para asegurar que las búsquedas y filtrados sean insensibles a mayúsculas, cumpliendo con los estándares de comunicación leal con el usuario.

2. DECISIONES TÉCNICAS Y ARQUITECTURAS

El sistema adopta una arquitectura de capas para separar la interacción con el usuario de la lógica de procesamiento:

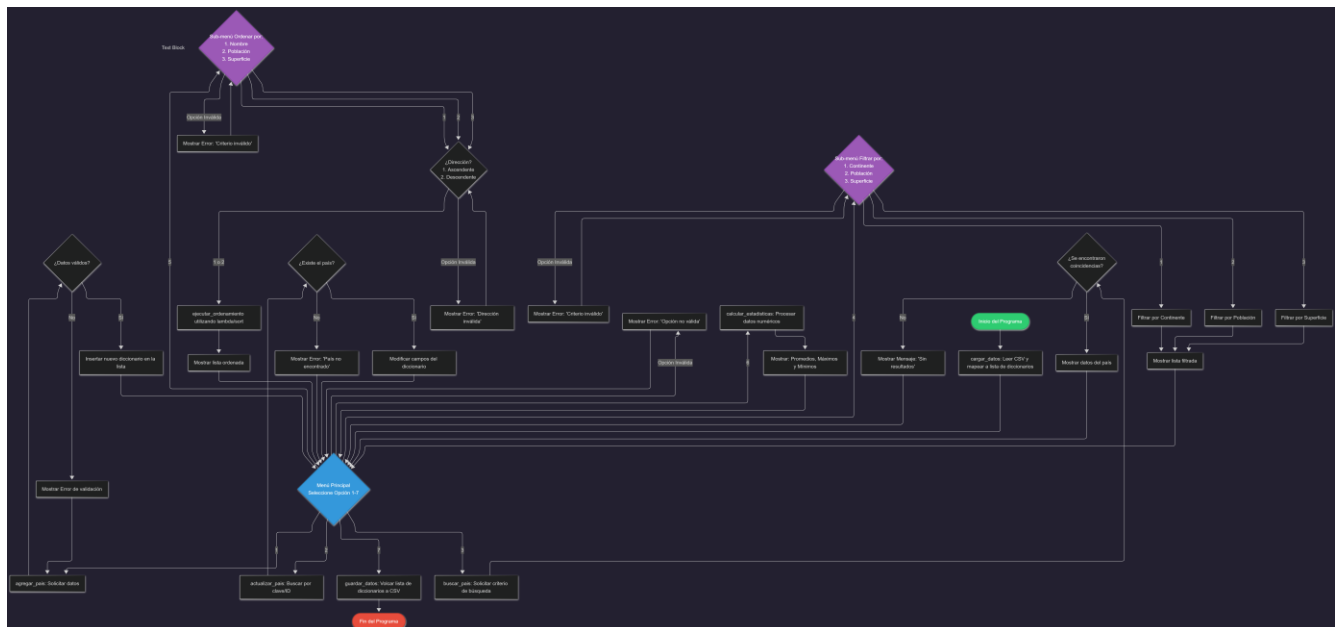
-**Bloque “2” de Validación (Funciones Auxiliares):** Se crearon funciones auxiliares (validar_entero, validar_texto, validar_continente) para sanitizar las entradas del usuario mediante bucles while True. Se destaca el uso de .strip() y .lower() para garantizar una comparación **case-insensitive** y la eliminación de espacios accidentales.

- **Bloque “3”- Lógica de Persistencia:** Las funciones cargar_datos y guardar_datos gestionan el flujo de información entre el archivo paises.csv y la memoria RAM, utilizando codificación UTF-8 para soportar caracteres especiales como tildes.

- **Bloque “4” – Lógica de Negocio:** Implementa las funcionalidades mínimas exigidas por la consigna. Se utilizó el operador in para permitir búsquedas y filtrados parciales, mejorando la usabilidad cuando el usuario no recuerda el nombre exacto del país.

- **Bloque “5”- Interfaz de Usuario (MAIN):** Se implementó un menú persistente basado en la estructura match-case para una selección de opciones limpia y eficiente.

A continuación, se exhibe un diagrama de flujo representativo del código py:



Para visualizar, hacer zoom o si se requiere, ingresar al siguiente link:

<https://mermaid.ai/d/6208a495-d734-4e52-93b5-e366cc489b20>

Diagrama de Flujo (Lógica General) – Paso a paso Para garantizar la robustez y la integridad de la información, el sistema sigue una secuencia de ejecución organizada en cinco etapas principales:	
1.Sincronización Inicial (Carga de Datos)	Al iniciar, el programa invoca de forma automática la función cargar_datos. Esta operación lee el archivo físico paises.csv y traslada toda la información a una lista de diccionarios en la memoria RAM, permitiendo que el sistema trabaje con la versión más actualizada del dataset desde el primer segundo.
2. Activación del Ciclo Interactivo	Una vez cargados los datos, el sistema entra en un bucle de persistencia (while). Este estado garantiza que el programa permanezca abierto y operativo, mostrando el menú principal de forma recurrente después de cada operación, hasta que el usuario decida finalizar la sesión explícitamente.
3.Captura y Validación de la Voluntad del Usuario	Cuando el usuario selecciona una opción, el sistema aplica un doble filtro de seguridad: Selección Modular: Mediante una estructura match-case, se direcciona el flujo hacia la función específica solicitada. Gestión de Errores (Robustez): Se utiliza un bloque try/except general en el núcleo del programa para capturar cualquier ingreso inválido o fallo inesperado, evitando que el software se detenga abruptamente ("explote") ante un error del usuario
4.Ejecución de Tareas Específicas	Según la opción elegida, se ejecutan las funciones de negocio (Alta, Actualización, Búsqueda, Filtrado, Ordenamiento o Estadísticas). Cada una de estas funciones es independiente y modular, lo que facilita el mantenimiento del código y asegura que cada tarea cumpla una única responsabilidad técnica.
5.Cierre Seguro y Persistencia Final:	Al seleccionar la opción de salida ("7"), el bucle principal se rompe y se activa la función guardar_datos. En este paso crítico, el sistema vuelca todo el contenido de la memoria RAM de nuevo al archivo paises.csv, garantizando que cualquier cambio, alta o modificación realizada durante la sesión se conserve de forma permanente en el disco duro antes de finalizar la ejecución

3. DIFICULTADES ENCONTRADAS Y RESOLUCIONES

A continuación, se exhibe un resumen de los principales desafíos técnicos que surgieron durante el desarrollo del software:

- **Problemas con uso de Caracteres Especiales:** Inicialmente, el sistema fallaba al buscar países con tildes (ej: "Japón") debido a que Python trata los caracteres acentuados como objetos distintos. Se resolvió migrando a comparaciones con el operador in. Asimismo, cuando era conveniente se indicaba al usuario usar acentos para los textos sí o sí al momento de la entrada de algún dato. Es decir, Japón se inserta sí o sí con acento, sin importar si escribe con minúscula o mayúscula en la primera letra, a fin de demostrar robustez.

- **Poca Robustez en el Ingreso de Continentes:** Se detectó que el sistema aceptaba cualquier texto en el campo continente. Se planteó implementar como constante al principio del código todas las opciones de nombres de continentes. Así, se implementó una **validación por lista blanca** (CONTINENTES_VALIDOS) que restringe el ingreso solo a opciones predefinidas, para asegurar la identificación de cada continente con solo escribir una parte del nombre (ej: "amer" para América).
- **Visualización de Datos Regionales:** La visualización inicial mostraba diccionarios crudos. Es decir, poco amigables para el usuario (Ej: población de Estados Unidos 349035494, en lugar de 349.035.494). De este modo, se refactorizó la capa de salida para aplicar formato de miles regional (puntos en lugar de comas) mediante f-strings y .replace(), aportando a la legibilidad del proyecto.
- **Lógica de Ordenamiento Dinámico:** Se superó la redundancia visual en los reportes mediante bloques de visualización condicionales que solo muestran el dato relevante al criterio de ordenación elegido por el usuario (Población o Superficie).

4. CONCLUSIONES

La realización de este proyecto permitió afianzar el uso de estructuras de datos complejas para la resolución de problemas reales. La transición de la memoria volátil a la persistencia en archivos CSV representa un salto cualitativo en la robustez de las aplicaciones, permitiendo que el software sea útil en escenarios productivos. Asimismo, el manejo de excepciones y la modularización demostraron ser herramientas indispensables para crear sistemas estables y fáciles de mantener por otros desarrolladores.

5. BIBLIOGRAFÍA

- Python Software Foundation. (2024). *The Python Tutorial: Input and Output*. Documentación oficial de Python 3.12. <https://docs.python.org/3.12/tutorial/inputoutput.html>
- Downey, A. B. (2015). *Pensar en Python: Aprenda a pensar como un informático*. (Trad. J. Espinoza). 2da Edición. Green Tea Press. <http://www.thinkpython2.com>
- Universidad Tecnológica Nacional (UTN). (2026). *Material de Cátedra: Programación I (Unidades 1 a 9)*. Sistema Institucional de Educación a Distancia (SIED). <https://sied.utn.edu.ar/>
- Git. (2024). *Reference Manual: Git Documentation*. <https://git-scm.com/doc>